AutoBrief: Automated Daily Briefing Bot

Full-Stack Application — Design, Architecture, and Implementation Guide

AutoBrief Project Documentation

March 2026

Contents

1 Introduction

AutoBrief is a full-stack web application that automates the generation and delivery of personalized daily briefings. It aggregates weather data, top news headlines, and a user's class schedule, then sends a formatted summary via Telegram at a configurable time each day.

1.1 Key Design Principles

- **Zero-cost external APIs**: Weather via Open-Meteo (free, no API key), news via Google News RSS (free, no API key). Only Telegram requires a free bot token from BotFather.
- **Self-contained**: SQLite database requires no external database server. APScheduler runs inside the application process.
- **Preview without Telegram**: The application supports a preview mode — users can see their briefing message without configuring Telegram.

2 System Architecture

The system follows a client-server architecture with three main layers:

1. **Frontend** (React + Vite + Tailwind CSS): Single-page dashboard for configuration and manual triggering.
2. **Backend** (Python + FastAPI): RESTful API server with CRUD endpoints, the briefing engine, and background scheduler.
3. **Data Layer** (SQLite + SQLAlchemy): Persistent storage for user preferences and class schedule.

2.1 Architecture Diagram

```
+-------------------+       HTTP/JSON        +-------------------+
|                   | <-------------------> |                   |
|   React Frontend  |   /api/* (proxied)     |  FastAPI Backend  |
|   (Vite :5173)    |                        |  (Uvicorn :8000)  |
|                   |                        |                   |
+-------------------+                        +--------+----------+
                                                      |
                                     +-------------------+-------------------+
                                     |                   |                   |
                                +------+------+    +-------+------+   +-------+-------+
                                |  SQLite DB  |    | Open-Meteo   |   | Google News   |
                                | (local file)|    | (free API)   |   | (RSS feed)    |
                                +-------------+    +--------------+   +---------------+
                                                      |
                                                +-------+-------+
                                                | Telegram Bot  |
```

```
                                          | API (free)    |
                                          +---------------+
```

2.2 Request Flow

When a user clicks "Preview Briefing":

1. React sends `POST /api/preview` to the backend.
2. FastAPI instantiates a `BriefingEngine`.
3. The engine reads user preferences from SQLite.
4. It concurrently fetches weather (Open-Meteo) and news (Google News RSS).
5. It reads today's class schedule from SQLite.
6. It builds a Markdown message and returns it in the JSON response.
7. React renders the message in a preview panel.

3 Directory Structure

```
AutoBrief/
  .env                        # Environment variables (Telegram token)
  .env.example                # Template for .env
  README.md                   # Quick-start guide
  autobrief_doc.tex           # This document
  backend/
    config.py                 # Loads env vars
    database.py               # SQLAlchemy engine, session, init
    models.py                 # ORM models (UserPreferences, ClassSchedule)
    briefing.py               # BriefingEngine (weather, news, Telegram)
    scheduler.py              # APScheduler cron job setup
    main.py                   # FastAPI app, routes, lifespan
    requirements.txt          # Python dependencies
    tests.py                  # Comprehensive test suite (23 tests)
    venv/                     # Python virtual environment
  frontend/
    index.html                # HTML entry point
    package.json              # Node.js dependencies
    vite.config.js            # Vite config with API proxy
    tailwind.config.js        # Tailwind CSS config
    postcss.config.js         # PostCSS config
    src/
      main.jsx                # React entry point
      index.css               # Tailwind directives
      App.jsx                 # Dashboard components
```

```
    node_modules/              # Node.js packages
```

Project Tree

4 Technology Stack

Layer	Technology	Purpose
Frontend	React 18	UI components and state management
	Vite 6	Build tool and dev server
	Tailwind CSS 3	Utility-first CSS framework
	Axios	HTTP client for API calls
Backend	Python 3.11+	Server-side language
	FastAPI 0.115	Async web framework with auto docs
	SQLAlchemy 2.0	ORM for database access
	APScheduler 3.10	In-process cron-like scheduler
	httpx 0.28	Async HTTP client
	feedparser 6.0	RSS/Atom feed parser
Database	SQLite	File-based relational database
External APIs	Open-Meteo	Weather data (free, no key)
	Google News RSS	News headlines (free, no key)
	Telegram Bot API	Message delivery (free token)

Table 1: Technology stack overview

5 Database Models

The application uses two SQLAlchemy ORM models backed by SQLite.

5.1 UserPreferences

Stores the user's configuration for the briefing engine. Only one row exists (singleton pattern — upserted on save).

Column	Type	Nullable	Description
id	Integer (PK)	No	Auto-increment primary key
city	String	No	City name for weather lookup
news_topics	String	No	Comma-separated news search terms
telegram_chat_id	String	No	Telegram chat ID for delivery

Table 2: UserPreferences schema

```python
class UserPreferences(Base):
    __tablename__ = "user_preferences"

    id = Column(Integer, primary_key=True, index=True)
```

```
    city = Column(String, nullable=False, default="London")
    news_topics = Column(String, nullable=False,
        default="technology")
    telegram_chat_id = Column(String, nullable=False,
        default="")
```

models.py — UserPreferences

5.2 ClassSchedule

Stores weekly class schedule entries. Multiple entries per day are supported, ordered by time.

Column	Type	Nullable	Description
id	Integer (PK)	No	Auto-increment primary key
day_of_week	String	No	Day name (Monday–Sunday)
time	String	No	24h time string (e.g., "09:00")
subject	String	No	Class/course name
location	String	No	Room or building (optional)

Table 3: ClassSchedule schema

6 Backend Implementation

6.1 Configuration (`config.py`)

Environment variables are loaded from a `.env` file using `python-dotenv`. The only required secret is the Telegram bot token (and even that is optional for preview mode).

```
import os
from dotenv import load_dotenv

load_dotenv(os.path.join(os.path.dirname(__file__), "..",
    ".env"))

TELEGRAM_BOT_TOKEN = os.getenv("TELEGRAM_BOT_TOKEN", "")
BRIEFING_HOUR = int(os.getenv("BRIEFING_HOUR", "7"))
BRIEFING_MINUTE = int(os.getenv("BRIEFING_MINUTE", "0"))
DATABASE_URL = "sqlite:///" + os.path.join(
    os.path.dirname(__file__), "autobrief.db"
)
```

config.py

6.2 Database Layer (`database.py`)

Uses SQLAlchemy 2.0 declarative base with a session factory. The `get_db` generator provides dependency-injected sessions to FastAPI route handlers.

```python
from sqlalchemy import create_engine
from sqlalchemy.orm import sessionmaker, DeclarativeBase
from config import DATABASE_URL

engine = create_engine(DATABASE_URL,
                       connect_args={"check_same_thread": False})
SessionLocal = sessionmaker(autocommit=False, autoflush=False,
                            bind=engine)

class Base(DeclarativeBase):
    pass

def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

def init_db():
    Base.metadata.create_all(bind=engine)
```

database.py

6.3 Briefing Engine (`briefing.py`)

The core service class that orchestrates data fetching and message construction.

6.3.1 Weather Fetcher — Open-Meteo

Uses a two-step process:

1. **Geocoding**: Resolves city name to latitude/longitude via `geocoding-api.open-meteo.com`.
2. **Forecast**: Fetches current weather using the coordinates via `api.open-meteo.com`.

WMO weather interpretation codes are mapped to human-readable descriptions (e.g., code 3 becomes "Overcast").

6.3.2 News Fetcher — Google News RSS

Fetches headlines by constructing a Google News RSS search URL with the user's topics, then parses the feed using the `feedparser` library. Article titles are split to extract the source name.

6.3.3 Message Builder

Constructs a Markdown-formatted message with three sections:

- Weather: city, temperature, humidity, wind speed, conditions

- Headlines: numbered list of top 3 articles with sources
- Schedule: today's classes with times and locations

Each section gracefully handles missing data (e.g., "unavailable" for weather failures).

6.3.4 Telegram Sender

Posts the formatted message to the Telegram Bot API using `httpx`. Returns early with a descriptive result if the bot token is not configured.

6.3.5 Orchestrator (`run` method)

The `run(preview_only=False)` method:

1. Reads preferences from the database.
2. Fetches weather and news (failures are caught individually).
3. Reads today's class schedule.
4. Builds the message.
5. If `preview_only=True` or no chat ID: returns the message without sending.
6. Otherwise: sends via Telegram and returns the result.

6.4 Scheduler (`scheduler.py`)

Uses APScheduler's `BackgroundScheduler` with a `cron` trigger. The job runs daily at the configured hour and minute (default: 07:00).

Since APScheduler uses synchronous jobs but the `BriefingEngine` is async, a wrapper function creates a new event loop to run the coroutine:

```python
def _run_briefing_sync() -> None:
    engine = BriefingEngine()
    loop = asyncio.new_event_loop()
    try:
        result = loop.run_until_complete(engine.run())
        print(f"[Scheduler] Briefing result: {result}")
    finally:
        loop.close()

scheduler = BackgroundScheduler()
scheduler.add_job(
    _run_briefing_sync,
    trigger="cron",
    hour=BRIEFING_HOUR,
    minute=BRIEFING_MINUTE,
    id="daily_briefing",
    replace_existing=True,
)
```

scheduler.py

6.5 FastAPI Application (`main.py`)

6.5.1 Lifespan Management

The application uses FastAPI's async context manager lifespan to:

- Initialize the database tables on startup.
- Start the APScheduler background scheduler.
- Shut down the scheduler on application exit.

6.5.2 CORS Middleware

Configured to allow requests from `http://localhost:5173` (the Vite dev server), with all methods and headers permitted.

6.5.3 API Endpoints

Method	Path	Description
GET	`/api/preferences`	Returns saved preferences (or null)
POST	`/api/preferences`	Creates or updates the singleton preferences row
GET	`/api/schedule`	Lists all schedule entries; optional `?day=Monday` filter
POST	`/api/schedule`	Creates a new schedule entry
PUT	`/api/schedule/{id}`	Updates an existing schedule entry
DELETE	`/api/schedule/{id}`	Deletes a schedule entry
POST	`/api/trigger`	Runs the full briefing pipeline (with Telegram send)
POST	`/api/preview`	Runs the briefing pipeline in preview mode (no send)

Table 4: REST API endpoints

7 Frontend Implementation

The frontend is a single-page React application built with Vite and styled with Tailwind CSS.

7.1 Development Proxy

The Vite configuration proxies all `/api/*` requests to the FastAPI backend at `http://127.0.0.1:8000`, eliminating CORS issues during development:

```
export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    proxy: {
```

```
        "/api": "http://127.0.0.1:8000",
      },
    },
  });
```

vite.config.js

7.2 Component Architecture

The `App.jsx` file contains four components:

1. **PreferencesForm**: Form with inputs for city, news topics, and Telegram chat ID. Loads existing preferences on mount, saves via POST.
2. **ScheduleManager**: Combined form and table for CRUD operations on class schedule entries. Supports inline editing and deletion.
3. **TriggerButton**: Two buttons — "Preview Briefing" (calls `/api/preview`) and "Send via Telegram" (calls `/api/trigger`). Displays the formatted message in a preview panel.
4. **App**: Shell component with header, three card-style sections, and footer.

7.3 State Management

Each component manages its own local state using React's `useState` and `useEffect` hooks. API calls use Axios with a base URL of `/api` (proxied to the backend).

8 External API Integration Details

8.1 Open-Meteo (Weather)

- **Cost**: Completely free, no API key required.
- **Geocoding endpoint**: `GET https://geocoding-api.open-meteo.com/v1/search?name={city`
- **Forecast endpoint**: `GET https://api.open-meteo.com/v1/forecast` with parameters for latitude, longitude, and current weather.
- **Data returned**: Temperature (Celsius), wind speed (km/h), WMO weather code, hourly humidity.
- **Rate limits**: Fair use (no hard limit for reasonable usage).

8.2 Google News RSS (News)

- **Cost**: Completely free, no API key required.
- **Endpoint**: `GET https://news.google.com/rss/search?q={query}&hl=en-US&gl=US&ceid=US`
- **Parser**: The `feedparser` library parses the RSS XML response.
- **Data returned**: Article title, source name (extracted from title suffix), and URL.

8.3 Telegram Bot API (Delivery)

- **Cost**: Free. Requires creating a bot via BotFather (`@BotFather` on Telegram).
- **Setup**: Message BotFather with `/newbot`, follow prompts, receive a token.
- **Chat ID**: Start a conversation with the bot, then use `https://api.telegram.org/bot{TOKEN}/ge` to find your chat ID.
- **Endpoint**: `POST https://api.telegram.org/bot{TOKEN}/sendMessage`

9 Test Suite

The project includes 23 automated tests in `tests.py`, organized into five test classes:

Test Class	Coverage	Count
TestModels	SQLAlchemy model creation	2
TestPreferencesAPI	GET/POST preferences CRUD	3
TestScheduleAPI	Full CRUD + edge cases	7
TestBriefingEngine	Weather, news, message builder, run	8
TestTriggerAPI	Trigger and preview endpoints	3
Total		**23**

Table 5: Test suite overview

9.1 Test Infrastructure

- Uses an in-memory SQLite database with `StaticPool` to share connections across threads.
- Each test gets fresh tables via an `autouse` fixture.
- FastAPI's dependency injection is overridden to use the test database.
- Integration tests hit the real Open-Meteo and Google News APIs (both free).
- `briefing.SessionLocal` is patched in orchestrator tests to use the test database.

Running the tests:

```
cd backend
source venv/bin/activate
python -m pytest tests.py -v
```

Running tests

10 Setup and Execution Guide

10.1 Prerequisites

- Python 3.11 or newer

- Node.js 18 or newer
- (Optional) A Telegram bot token from BotFather (free)

10.2 Environment Configuration

Edit the `.env` file in the project root:

```
TELEGRAM_BOT_TOKEN=your_telegram_bot_token
BRIEFING_HOUR=7
BRIEFING_MINUTE=0
```

.env

The Telegram token is optional — the app works in preview mode without it.

10.3 Backend Setup

```
cd backend
python3 -m venv venv
source venv/bin/activate        # macOS/Linux
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Backend setup commands

The API will be available at `http://127.0.0.1:8000`. Interactive API docs are at `http://127.0.0.1:8000/docs`.

10.4 Frontend Setup

```
cd frontend
npm install
npm run dev
```

Frontend setup commands

The dashboard will be available at `http://localhost:5173`.

10.5 Usage Workflow

1. Open `http://localhost:5173` in a browser.
2. Fill in preferences: city name, news topics, and (optionally) Telegram chat ID.
3. Click "Save Preferences".
4. Add class schedule entries using the schedule form.
5. Click "Preview Briefing" to see the formatted message.
6. (With Telegram configured) Click "Send via Telegram" to deliver it.
7. The scheduler automatically sends the briefing daily at the configured time.

11 Sample Briefing Output

Below is an actual briefing message generated by the system during testing:

```
*Daily Briefing -- Monday, March 16, 2026*

*Weather*
City: New York
Temp: 15.0 C
Humidity: 69%
Wind: 26.2 km/h
Conditions: Overcast

*Top Headlines*
1. Zelenskiy says Ukraine wants money, technology
   in return for Middle East drone help (Reuters)
2. What Iran's war reveals about future of defense
   technology (The Jerusalem Post)
3. Gorilla Technology & Yotta Sign Landmark AI
   Infrastructure Deal (Gorilla Technology)

*Today's Classes (Monday)*
- 09:00  Data Structures -- Room 302

_Have a productive day!_
```

Sample briefing (March 16, 2026)

12 Conclusion

AutoBrief demonstrates a complete full-stack application with the following characteristics:

- **Zero-cost operation**: All external APIs (Open-Meteo, Google News RSS) are completely free with no API keys required. Only Telegram needs a free bot token.
- **Self-contained deployment**: SQLite eliminates database server dependency. APScheduler runs in-process.
- **Comprehensive testing**: 23 automated tests covering models, API endpoints, the briefing engine, and integration with external APIs.
- **Graceful degradation**: Individual API failures are caught and the briefing still generates with available data.
- **Preview mode**: Full briefing generation without Telegram configuration for development and testing.